Appendix

A Worked Example: Laws 12857 and 33436

References to sections, tables and figures without an A-prefix refer to the main paper, and citations resolve in the main paper's reference list.

Among the laws the portfolio resolves is a pair the Equational Theories Project (Bolan et al. 2025) recorded as having only trivial finite models, with the existence of an infinite model left unresolved. In the ETP numbering they are laws 12857 and 33436:

$$L_{12857}: \quad x = y \diamond ((x \diamond (y \diamond (z \diamond z))) \diamond y),$$
$$L_{33436}: \quad x = ((y \diamond (((z \diamond z) \diamond y) \diamond x)) \diamond y).$$

The two are duals: reversing the arguments of every occurrence of $\diamond$ in one yields the other. A magma satisfies one exactly when its opposite magma satisfies the other, so the pair is a single open case up to duality, and resolving either resolves both.

A.1 Two independent certificates

Both laws were resolved twice, by methods that share no search strategy.

Vampire 5.0.1 (Kovács and Voronkov 2013) was run in ordered saturation mode (`-sa otter`, default Knuth–Bendix ordering) on the law together with the assertion that two elements are distinct. It terminated with `SZS status Satisfiable` in 7.95 seconds and 126 MB of memory, printing an active set of 357 unit equations. A terminating, complete saturation containing no contradiction describes a model, as set out in Section 3.3; here the model is one in which $x = y$ fails, and the admissibility certificate the instance already carries forces that model to be infinite.

Twee 2.6.1 (Smallbone 2021) was run independently on both laws and returned `SZS status CounterSatisfiable` for each, deriving 55 rules for 12857 and 54 for 33436. Unfailing completion is complete by construction, so this is a second certificate for the same conclusion by a different route.

A.2 Verifying the presentation

The saturated presentation is checked directly rather than taken on the prover's word. Three properties are confirmed. The ordering check verifies that the ordering used to evaluate the presentation matches the one recorded in the certificate header, which is necessary because ground confluence is inherited from the saturation only with respect to the ordering the prover saturated under; every pre-ordered equation is comparable under that ordering. The law then holds on every ground instance tested, 27 of 27, and rewrites fire in all of them, so the model is not satisfying the law vacuously. Finally the two Skolem constants of the distinctness axiom normalise to different terms, which establishes that the carrier has at least two elements.

The presentation is too large to reproduce here; the certificate ships in the supplementary material, and `ordered_model.py` reproduces all three checks from it.

A.3 Why this model is not Lean-checkable today

The presentation is not an off-the-shelf term rewriting system, which is why this model is graded by the prover certificate rather than by the Lean kernel (Moura and Ullrich 2021). Of the 357 equations, a substantial minority carry a free variable on each side, so they cannot be oriented into rewrite rules in the usual sense.

That is a constraint on printing the system, not on the construction. The model rewrites a ground term whenever the ground instance strictly decreases, and a variable unbound on the other side of an equation may be mapped to any ground term that makes the instance decrease — in practice the smallest constant. Every equation is therefore usable, in whichever direction decreases the instance. Termination is immediate because the ordering is well founded on ground terms, and ground confluence is inherited from saturation under unfailing completion.

What is missing is an off-the-shelf certifier. Tools such as CSI and TTT2 check plain rewriting systems, and the object here is an ordered one. A Lean formalization of this model therefore awaits a machine-checked ground confluence proof. Models of the algebraic kind, by contrast, are Lean-checkable today; no admissible law admits one, for the reason given in Section 3.3.

B LLM Evaluation Protocol and Prompts

B.1 The fixed protocol

Every solver receives the same three inputs: the law, the task statement, and a fixed answer format. A submission may be sent to the judge up to three times, and each verdict is returned verbatim as feedback before the next attempt. No solver receives a hint that is not derived mechanically from the law itself, and the harness is frozen across all language models, so differences in outcome are attributable to the model rather than to the scaffolding.

Saturation provers are withheld from the language-model setting. With access to them a solver could re-run the automated portfolio of Section 4.1 rather than construct anything; the judge and the autoformalization harness are the tools available to any solver built on ALPS.

All LLM calls use provider-default sampling through a single API gateway. The APIs expose no seed parameter, and the reasoning LLMs are nondeterministic at any setting, so bitwise reproduction of a run is not available. Run-to-run variation is therefore measured rather than controlled: the reproduction run of Section 4.2 repeats the nine solved and five matched unsolved laws under the identical frozen harness, and the result files in the supplementary material record every attempt verbatim.

B.2 The trivial channel

On the trivial side the formalization burden is removed. The model emits a chain of equalities between two arbitrary carrier elements a and b, each step being one application of the law, and returns it as JSON. The harness matches each step against the law, bridges gaps of at most three single applications automatically, and assembles the Lean proof itself. The

model therefore searches for a derivation rather than writing Lean.

A step is accepted only when some substitution instance of one side of the law rewrites one term into the next at exactly one position, with everything outside that position unchanged. Two rejections are absolute: relating a and b directly, and any jump the bridging search cannot close.

A *waypoint lemma* is an equality a prover has already derived from the law, supplied in the prompt. Waypoints shorten planning — the model can build a chain to a waypoint and continue from there — but they cannot repair an individual step, since every step the model writes must still be a correct application of the law. Comparing runs with and without waypoints therefore separates failures of chain-finding from failures of step validity.

B.3 The construction channel

On the construction side the model proposes a finite presentation E: a set of unconditional equations over $\diamond$, returned as JSON. The submission format admits only unconditional equations, so implications, disequations and literal restatements of $x = y$ never reach the prover. The judge then runs the two checks of Equation (6).

B.4 Prompts

Figure A1 gives the trivial-channel prompt and Figure A2 the construction-channel prompt, both verbatim. Braces marked `{law}` and `{hint_block}` are substituted at call time; the hint block is empty in the no-waypoint condition and lists the prover-derived waypoint lemmas otherwise. On a rejected submission the harness appends the judge's reason to the next round's prompt: `Your previous chain was REJECTED: {feedback}` on the trivial side, and `Your previous proposal failed to certify: {feedback}` followed by `Try a different presentation.` on the construction side.

C Automated Baseline: Per-Configuration Results

The portfolio spans eight configurations: Vampire 5.0.1 in five modes, E 3.3.5 (Schulz, Cruanes, and Vukmirović 2019) in two, and Twee 2.6.1. Table A1 reports how many distinct laws each configuration resolves at each per-configuration budget, over the 4,141-law residual. The counts overlap, since several configurations resolve the same law, so the column totals exceed the 114 laws resolved overall.

Two features of the table bear on the main paper's claims. First, every configuration concentrates its resolutions at the 30-second rung; no configuration shows a profile in which added budget yields resolutions at a steady rate. Second, all four models the submitted sweep recovers come from a single configuration, `twee/complete`. The Vampire saturation modes resolve 14 laws each, all by proving triviality. The construction side of the portfolio therefore rested on one configuration.

The sweep exits a law's ladder as soon as any configuration resolves it, so the number of laws reaching each rung declines: 4,141 at 30 seconds, then 4,050, 4,042, 4,037 and 4,033.

D LLM Failure Analysis

Each language model attempts all 25 hard-tier laws over three feedback rounds, and no submission from any model passes both checks. Because a run ends either in acceptance or after three rejections, every law leaves a final-round submission, and Table A2 classifies all 75 of them by which of the two checks that final submission satisfies.

The three classes partition the outcomes. A presentation is *too strong* when it entails the law but admits only one-element models: check (a) succeeds and check (b) fails. It is *too weak* when it keeps two elements distinct but does not entail the law: check (b) succeeds and check (a) fails. A submission failing both satisfies neither condition. The final column, counting laws on which the model clears exactly one check, is therefore 25 minus the both-fail count.

The trivial-side failures follow one dominant pattern at every setting, and it is a different kind of failure. The submitted chains contain steps that are not applications of the law: no substitution instance of either side rewrites one term into the next, so the harness rejects the chain at the matching stage rather than at the prover. The failure is in executing individual steps soundly, not in planning a route between a and b.

E Corpus Yield and Renewability

Section 3.2 claims that the corpus has no terminal order and that fresh instances can therefore be minted after any training cutoff. That claim is worth checking against yield: a generator whose output thins as the order grows would eventually stop supplying instances, whatever the recursion permits in principle.

Table A3 gives the yield of the screening pipeline broken down by the order of the candidate law, over all 10,474 candidates screened.

Yield must be read as a density rather than a count. The raw number of Austin laws dips from 117 at order 7 to 105 at order 8, but fewer order-8 candidates were screened, so the dip measures how much was generated rather than how much was there. Normalised to admissible laws per thousand screened candidates, the yield rises across every order the extension engine produces: 20.5 at order 6, 24.4 at order 7, and 25.4 at order 8. Order 5 is excluded from the comparison because those laws are ETP's curated enumeration rather than a uniform draw from the generator, which is why its density is several times higher.

Counting only laws that open a new equivalence class gives 16.9, 19.4 and 16.2 per thousand across the same three orders. That density does not fall, so the corpus continues to produce genuinely distinct problems rather than variations on classes it has already covered; unlike the overall yield, it is not monotonic, and we do not claim it is rising.

F Validation of the Automated Judge

Both channels of the judge run a control suite before any evaluation. The suites are part of the released code and can be re-run with `--selftest`.

Configuration	30	60	120	300	600	Total	Models
twee/complete	69	5	4	3	5	86	4
eprover/triv/auto	64	2	2	1	1	70	0
vampire/triv/casc+lpo	30	1	1	1	0	33	0
vampire/triv/casc	24	1	0	1	0	26	0
vampire/triv/discount	22	0	0	0	0	22	0
vampire/sat/otter+lpo	13	1	0	0	0	14	0
vampire/sat/otter+kbo	12	1	1	0	0	14	0
eprover/sat/satauto	0	0	0	0	0	0	0

Table A1: Distinct laws resolved by each configuration at each per-configuration budget in seconds, over the 4,141-law residual. Counts overlap across configurations. "Models" counts laws resolved by exhibiting a nontrivial model rather than by proving triviality.

LLM	Too strong	Too weak	Both fail	One check
o3	23	2	0	25
o4-mini	11	4	10	15
GPT-4.1	10	11	4	21

Table A2: Final-round classification of all 25 hard-tier construction submissions per LLM. "Too strong" entails the law but admits only one-element models (passes check (a), fails (b)); "too weak" maintains two distinct elements but fails to entail the law (passes (b), fails (a)). The last column counts laws on which the LLM clears exactly one of the two checks.

Order	Screened	Austin	Austin per 1,000
5	115	11	95.7‡
6	1,418	29	20.5
7	4,803	117	24.4
8	4,138	105	25.4

Table A3: Admissible-law yield by order, over all 10,474 screened candidates. Density rises monotonically across the orders the extension engine generates, so the supply does not thin as the corpus grows. ‡Order 5 is excluded from the trend: those laws come from ETP's curated enumeration rather than a uniform draw from the generator.

The trivial channel. A submission is a Lean proof of a proposition the judge generates, and two things must hold: the proof must elaborate at exactly the generated type, and its axiom footprint must lie within the allowlist {`propext`, `Quot.sound`, `Classical.choice`}. Anything outside that set is a failure, and `sorryAx`, `Lean.ofReduceBool` and `Lean.trustCompiler` are named explicitly as forbidden. The footprint is an allowlist rather than a blocklist so that an unanticipated axiom fails closed.

Seven negative controls exercise the ways a submitter could avoid proving anything, and each is rejected by a textual scan that runs before Lean is invoked, since several of them would otherwise subvert Lean itself: a `sorry`; a declared axiom asserting the goal; two forms of namespace shadowing; redefining the law predicate as `True`; `native_decide`; and naming the submitted theorem anything other than the identifier the judged file elaborates. A positive control confirms the suite is not simply rejecting everything: a reference answer passes the scan and compiles with an allowed footprint.

The construction channel. Three controls cover the three ways a presentation can fail to encode a construction, matching the cases discussed in Section 3.3. Submitting the law itself serves as the positive control: it certifies exactly when bare saturation of $L \cup \{\, a \neq b \,\}$ terminates, which screening has already established for the seed law used here. On the hard tier the same submission diverges at check (b), so echoing the law back does not produce a certificate there. The left projection $\{x \diamond y = x\}$ is too weak: it admits models with distinct elements but does not entail the law, so check (a) fails. The collapse $\{x = y\}$ is too strong: it entails the law but admits only one-element models, so check (b) fails. The suite passes only when the first certifies and the other two do not.

G Diversity of the Constructed Models

Section 3.2 reports corpus size in equivalence classes, so that laws posing the same problem are not counted twice. Logical equivalence is the natural criterion, but it leaves a question open: two laws that are not equivalent may still be satisfied by essentially the same construction, in which case the class count would overstate how many distinct problems the corpus contains.

We test this directly, without a prover, by asking which laws each constructed model satisfies. Every model built during screening is evaluated against every one of the 262 laws proved Austin, giving 68,382 ordered pairs and no missing models. A model transfers to a law when it satisfies that law as well as its own.

The result is that construction classes and logical classes coincide exactly: both number 195. The pairs on which transfer runs in both directions number 255, precisely the 255 pairs the prover proved equivalent, so mutual model-satisfaction and logical equivalence pick out the same relation — a cross-check on the deduplication of Section 3.2 by a method that shares none of its machinery. Transfer across class boundaries is rare and one-directional: 173 of 68,382 ordered pairs,

or 0.25%. A typical model satisfies only the law it was built for, with a median coverage of one law, a mean of 3.67 and a maximum of 19. No cell is undecided, and every model satisfies its own law.

The 195 classes are therefore construction-distinct and not merely logically distinct. The models do not collapse onto a shared construction that a solver could learn once and reapply.

H Computing Infrastructure and Cost

All prover experiments ran on a dual-socket AMD EPYC 7313 machine — 2 sockets $\times$ 16 cores with SMT disabled, giving 32 physical cores — with 503 GB of RAM under Ubuntu 22.04.5 LTS. The portfolio sweep was sharded 32 ways, one shard per physical core, so no configuration competed with a sibling hyperthread for its time budget. Language-model experiments were run through the OpenRouter API.

Table A4 reports token usage for every language-model run described in the paper. The two certified-easy runs account for more than half of the total, because each of the 63 laws may consume up to three feedback rounds and because reasoning traces on that set are long: a solved law costs a median of 49,221 completion tokens. The construction runs are cheaper per law despite resolving nothing, since a presentation is a short object to emit. GPT-4.1's totals are an order of magnitude below the reasoning models' at comparable call counts, which reflects the absence of a reasoning trace rather than a shorter answer.

Files with more than 63 records contain repeated passes under identical settings, appended in run order. The no-waypoint o3 file holds a first pass over all 63 laws, on which four chains certify, followed by a second pass over the 36 laws then unsolved, which adds two more; the six no-waypoint solutions reported in the main text are the union of the two passes. The waypoint o3 file holds a first pass over all 63 laws, on which nine chains certify, followed by two further passes over 32 of the unsolved laws, which add none. The reproduction run is a separate file.

Run	Records	Completion	Prompt
o3 cert-63, waypoints	127	3,094,531	164,044
o3 cert-63, no waypoints	99	3,289,573	130,963
o3 low effort	9	152,309	20,724
o3 reproduction	14	575,997	59,653
o4-mini cert-63	63	974,972	147,300
GPT-4.1 cert-63, waypoints	63	61,670	149,538
GPT-4.1 cert-63, no waypoints	63	71,736	139,620
GPT-4.1 pilot (10 laws)	10	9,234	31,870
o3 trivial, hard-25	25	1,226,978	53,475
o3 construct, hard-25	25	1,120,628	144,122
o4-mini construct	25	554,835	24,818
GPT-4.1 construct	46	6,605	45,755
Total	569	11,139,068	1,111,882

Table A4: Token usage across all reported language-model runs. A solved certified-easy law costs a median of 49,221 completion tokens.

```
You are proving a magma law forces triviality. You will NOT write Lean --you
give only a sequence of terms, and a theorem prover checks each step.

Law: {law}
Read it as a rewrite rule. Write the law as L = R, where L is the single
variable on the left and R is the big right-hand term. ONE step does exactly
one of:
  •pick any subterm `e` of the current term and replace that ONE occurrence
    with R, taking L := e (so `e` becomes R with the law's left variable set
    to `e`); OR
  •the reverse --replace a subterm that exactly matches an instance of R by
    the corresponding `e`.
Everything OUTSIDE that one chosen subterm stays byte-for-byte identical.

Goal: build a = t1 = t2 = ... = b. Adjacent terms should be only a FEW (1-3)
such steps apart --you may skip intermediate terms and the checker will fill
short gaps automatically, so give WAYPOINTS, not necessarily every atomic
step. Prefer smaller jumps.

HARD RULES --a jump breaking any of these is rejected, so self-check each one
before writing it:
  •You may NOT write "a = b" or otherwise relate a and b directly. They are
    opaque atoms; the only way to connect them is through the law.
  •Each jump must be a GENUINE short derivation --at most ~3 single law
    applications apart. A leap between unrelated terms cannot be bridged and
    is rejected.
  •Every application is a REAL instance: a subterm `e →R[L:=e]` (or its
    reverse) for some choice of the law's other variables. If you can't name
    that instance, the step is invalid.

STRATEGY: rewrite FORWARD from `a` (each rewrite grows the term via the law)
toward a term the law forces to collapse --these laws let you derive
`op(_, _) = (any element)`; once both `a` and `b` reduce to a common term, the
chain closes.{hint_block}

Return ONLY JSON: {"chain": ["a", "<term>", ..., "b"]} first "a", last "b";
use only a, b, ◇, ().
Example (toy law x = y ◇y): {"chain": ["a", "(a ◇a)", "b"]}
  step 1: `a` →`(a ◇a)` --the law with x:=a, y:=a gives a = a◇a. ✓
  step 2: `(a ◇a)` →`b` --reverse: the law with x:=b, y:=a gives b = a◇a,
    so a◇a = b. ✓
Only write steps you can justify this way.
```

Figure A1: The trivial-channel prompt, verbatim. The model returns a chain of terms as JSON; the harness matches each step against the law, bridges gaps of at most three single applications, and assembles the Lean proof.

```
You are constructing an infinite model for a magma law, to be checked by an
automated theorem prover.

The law is: {law}
(the binary magma operation is written with the diamond; the law holds for all
inputs.)

This law has NO nontrivial FINITE model, so its model is necessarily infinite,
and it CANNOT be an arithmetic formula (any polynomial op over the integers
would descend to a finite model). Instead, propose a finite set of DEFINING
EQUATIONS for the operation --a presentation of the model --from which the law
follows and which is consistent with the carrier having two distinct elements.

Think of it as the completed rewrite system a Knuth-Bendix procedure would
converge to: a handful of equations that (a) entail the law, and (b) do not
force all elements equal.

Return ONLY a JSON object, nothing else:
  {"E": ["<equation>", "<equation>", ...]}
Each equation uses variables x,y,z,w and the diamond operator, in the form
LHS = RHS .
Example shape (NOT a solution):
  {"E": ["x ◇(y ◇x) = y", "(x ◇x) ◇x = x ◇x"]}
Do NOT just restate the law, and do NOT write `x = y`.
```

Figure A2: The construction-channel prompt, verbatim. The model returns a finite presentation E as JSON, which the judge submits to the two checks of Equation (6).